ROS2 Fundamentals

Dinesh Varma Bhetalam
Manipal Institute of Technology,
Manipal, Karnataka, India

***Abstract*—This report explains the main ideas behind ROS 2: the computation graph, nodes, topics, services, actions, parameters, custom interfaces, the colcon workspace and QoS. It compares topics with services and says when to use each. Two small exercises then apply these ideas: a wheel-speed publisher and subscriber, and a three-node palindrome checker. Terminal output and `rqt_graph` views are included.**

***Index Terms*—ROS 2, publish–subscribe, services, QoS, colcon, custom messages.**

I. INTRODUCTION

ROS 2 is a framework for building robot software as many small programs that exchange data . It runs on the DDS standard , so no central master is needed. This report covers the core concepts (Sections II–VI) and two exercises on a ROS 2 Humble workspace (Sections VII and VIII).

II. THE COMPUTATION GRAPH

The computation graph is the network of everything exchanging data in a running ROS 2 system. Nodes are the points, and topics, services, actions and parameters are the links. Nodes discover each other automatically, so nothing has to be wired up by hand. The tool `rqt_graph` draws the result, with ellipses for nodes and rectangles for topics.

A. Nodes

A node is one program with one job, such as reading a sensor or checking a string. Small nodes are easy to reuse and restart, and one failing does not stop the rest. A typical robot has dozens of them: one per sensor, one per controller, one for planning, and so on.

B. Topics

A topic is a named channel for one-way messaging. Publishers send, subscribers receive, and neither needs to know about the other. Topics suit data that keeps flowing, like sensor readings. Several nodes can listen to the same topic, and a node can start or stop listening at any time without the sender changing.

C. Services

A service is a request followed by one reply. A single server answers, and the client waits. Services suit quick, occasional jobs such as “reset the odometer”. Because the caller waits, a service should return fast.

D. Actions

An action is for long jobs, like “drive to this point”. The client gets progress updates and can cancel midway. Internally it combines services (send the goal, cancel, get the result) and topics (progress and status).

E. Parameters

Parameters are settings that belong to a node, such as a wheel radius or a gain. They can be set at start-up or changed while the node runs, which means a node can be tuned without being rewritten.

III. TOPICS VERSUS SERVICES

TABLE I. TOPICS AND SERVICES COMPARED

	Topic	**Service**
Pattern	Publish–subscribe	Request–reply
Who talks	Many to many	One server, many clients
Timing	Continuous stream	On demand
Reply	None	Always one
Examples	Speed, camera, odometry	Reset, compute once

Use a *topic* when data flows all the time, several nodes may want it, or the sender should not depend on any receiver. Use a service when the caller needs an answer before it can continue. Use an *action* when the job is slow and needs progress or cancellation.

A simple test: if a listener that joins late can safely ignore old values because a newer one is coming, it is a topic. If the caller cannot carry on until it hears back, it is a service.

Our palindrome exercise sits right on this line. Node 3 has to answer Node 2, which sounds like a service. But the strings are published continuously and the answers are just as continuous, so we used a second topic. This keeps the nodes independent, and it shows up as a loop in the graph.

IV. CUSTOM INTERFACES

When the standard message types do not fit, ROS 2 lets you define your own message, service and action files. The build tool turns them into code that nodes can import. Field names must be lowercase with underscores, so `angVel` becomes `ang_vel`. Interfaces are best kept in their own package so that many node packages can share them, as we did in both exercises. Forgetting to build the interface package first is the most common reason a new message cannot be imported.

V. WORKSPACES AND COLCON

A workspace is a folder holding packages. Its `src` folder contains the source, while `build`, `install` and `log` are generated. A package can be Python or C++ and lists its dependencies in `package.xml`. colcon builds all packages in the right order, so an interface package is always built before the nodes that use it. After building, the workspace is “sourced” in each new terminal so ROS can find it. Forgetting this step is the usual cause of a “package not found” error.

VI. QUALITY OF SERVICE

QoS settings let each connection trade reliability against speed. Table II lists the main ones.

TABLE II. MAIN QOS POLICIES

Policy	Options	Effect
Reliability	Reliable / Best effort	Resend lost data or not
Durability	Volatile / Transient local	Late joiners get old data or not
History	Keep last N / Keep all	How much data is stored

The default is reliable, volatile, keep last 10, which suits low-rate data we cannot lose. Fast streams like lidar often use best effort instead. Two ends connect only if their settings are compatible: a best-effort publisher cannot serve a reliable subscriber. Both exercises use the defaults. When two nodes refuse to connect even though names and types match, a QoS mismatch is the first thing to check.

VII. Question 1: WHEEL SPEED

This exercise uses a custom message `TaskOne` holding the angular velocity and radius of a wheel. The field is named `ang_vel`, because capital letters are not allowed in message fields. A publisher takes both values from the terminal and sends them on `/taskone`. A subscriber receives them and prints the longitudinal speed, given by

$$v = \omega \cdot r \qquad (1)$$

with v in m/s, ω in rad/s and r in m. Entering ω = 2, r = 3 and then ω = 4, r = 5 gave 6 m/s and 20 m/s (Fig. 1), matching a hand calculation (Table III).



Fig. 1. Subscriber printing the computed speed.

TABLE III. WHEEL SPEED RESULTS

ω (rad/s)	r (m)	Hand v (m/s)	Node v (m/s)
2	3	6	6
4	5	20	20

Fig. 2 shows the graph: publisher, then `/taskone`, then subscriber. The other two ellipses are not part of the design: `/minimal_service` was left running from an earlier exercise, and the last one is `rqt_graph` itself.

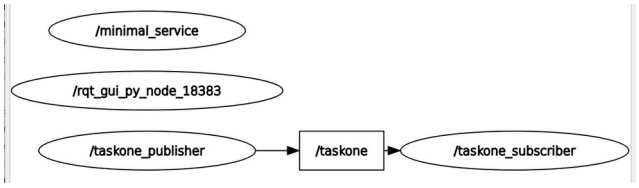


Fig. 2. rqt_graph for Case Study 1.

VIII. Question 2: PALINDROME CHECK

Three nodes work together, using a custom message `RT2String` that holds one string. Node 1 publishes a typed string over and over. Node 2 checks whether it is a palindrome and sends 1 or 0 to Node 3. Node 3 answers “Yes” or “No”, and Node 2 prints the answer. Since Nodes 2 and 3 talk both ways, the reply travels on a second topic. Only the first topic uses the custom message; the other two use standard integer and string types, which saves defining interfaces we do not need.

Node 1 has to do two things at once: wait for the user to type and keep publishing. Reading the keyboard in a separate thread solves this, and a timer repeats the latest string every second. Typing a new string simply replaces the old one. The palindrome check compares the text with its reverse and is case-sensitive.

We typed “AABAA”. Node 1 repeated it every second (Fig. 3), Node 2 sent 1 and printed the reply “Yes” (Fig. 4), and Node 3 confirmed it received 1 and answered “Yes” (Fig. 5). Fig. 6 shows the loop between the checker and the responder.

TABLE IV. NODES IN CASE STUDY 2

Node	Listens to	Publishes
1 rt2_publisher	Keyboard input	/rt2_string
2 rt2_checker	/rt2_string, /palindrome_reply	/palindrome_result
3 rt2_responder	/palindrome_result	/palindrome_reply

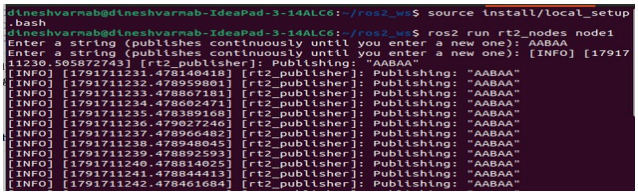


Fig. 3. Node 1 publishing “AABAA”.

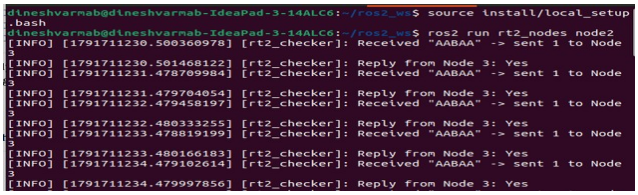


Fig. 4. Node 2 checking and printing the reply.

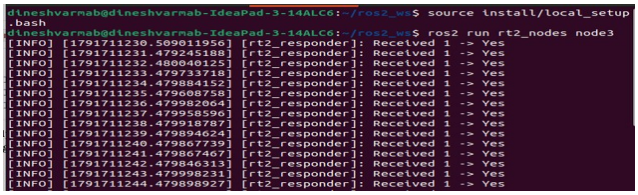


Fig. 5. Node 3 answering “Yes”.

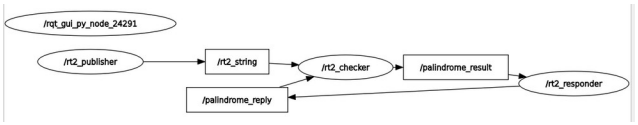


Fig. 6. rqt_graph for Case Study 2.

IX. CONCLUSION

Three lessons stood out. Message field names cannot use capitals. A node that waits for keyboard input cannot also process callbacks, so the two jobs must be separated. And every new terminal must source the workspace before running anything.

Topics carry continuous data, services give one answer, actions handle long jobs, and parameters hold settings. A small interface package plus a node package, built with colcon, is enough to connect nodes with a custom message. Both exercises gave correct results, and their graphs matched the designs.

REFERENCES

[1] https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools.html

[2] Open Robotics, ROS 2 Documentation: Humble Hawksbill. https://docs.ros.org/en/humble/